

Работа с git в Neovim

Работа с Git в Neovim

 Быстрые ссылки открывают PDF на GitHub (требуется интернет)

Быстрые ссылки

-  [Все плагины \(главная\)](#)
 -  [Файловый менеджер](#)
 -  [LSP и автодополнение](#)
 -  [UX-плагины](#)
-

Подробный гайд по работе с Git в Neovim

Зачем нужен Git?

Git — это современная система контроля версий, которая позволяет:

- Хранить историю изменений кода
- Работать в команде и не терять свои наработки
- Безопасно экспериментировать с ветками
- Откатываться к любому состоянию проекта
- Сливать, сравнивать, анализировать изменения

В Neovim git интегрируется через плагины, что делает работу с кодом ещё удобнее и быстрее.

Плагины для Git в Neovim

gitsigns.nvim

Показывает изменения (hunks) прямо в редакторе:

- `]h` / `[h` — переход к следующему/предыдущему изменению
- `<leader>hs` — добавить изменение в индекс (stage hunk)
- `<leader>hr` — откатить изменение (reset hunk)
- `<leader>hp` — показать diff
- `<leader>hb` — blame для строки

Совет: Используй визуальный режим для stage/reset нескольких строк!

lazygit.nvim

Мощный git-UI прямо в Neovim:

- `Space` + `lg` — открыть lazygit
- Просмотр истории, stage/unstage файлов и hunks, merge, rebase, push, pull — всё через удобный интерфейс

Основные команды Git

Команда	Описание
git init	Инициализация репозитория
git clone URL	Клонировать репозиторий
git status	Текущее состояние файлов
git add file	Добавить файл в индекс (stage)
git commit -m "msg"	Зафиксировать изменения (commit)
git log	История коммитов
git diff	Показать различия
git branch	Список веток
git checkout branch	Переключиться на ветку
git merge branch	Слить ветку
git pull	Получить и влить изменения с сервера
git push	Отправить изменения на сервер
git stash	Спрятать текущие изменения
git tag v1.0	Создать тег
git reset --hard HEAD^	Откатить состояние на коммит назад
git rebase branch	Перенести коммиты поверх другой ветки
git cherry-pick hash	Взять отдельный коммит в текущую ветку
git revert hash	Откатить отдельный коммит

Практические примеры команд

1. Клонирование репозитория

```
git clone https://github.com/username/project.git
```

2. Создание новой ветки и переход в неё

```
git checkout -b feature/my-feature
```

3. Добавление и коммит изменений

```
git add .  
git commit -m "Добавил новую фичу"
```

4. Слияние ветки (merge)

```
git checkout main  
git merge feature/my-feature
```

5. Перебазирование (rebase)

```
git checkout feature/my-feature  
git rebase main
```

6. Откат последнего коммита (оставить изменения в рабочей директории)

```
git reset --soft HEAD~1
```

7. Восстановление утерянного коммита

```
git reflog  
git checkout <hash>
```

8. Быстрое исправление последнего коммита

```
git commit --amend -m "Новый текст коммита"
```

9. Спрятать и вернуть незакоммиченные изменения

```
git stash  
git stash pop
```

10. Просмотр, кто изменил строку

```
git blame main.lua
```

Советы и лайфхаки для реальной работы

- **Работай в отдельных ветках** для каждой задачи/фичи — это упростит ревью и откат.
- **Перед слиянием всегда делай** `git pull --rebase` — так история будет чище.
- **Не бойся экспериментировать:** если что-то пошло не так — всегда есть `git reflog` и `git reset`.
- **Используй теги** для релизов: `git tag v1.0` — удобно возвращаться к стабильным версиям.
- **gitsigns:**
 - Используй визуальный режим для `stage/reset` сразу нескольких строк.
 - Быстро смотри `diff` (`<leader>hp`) и `blame` (`<leader>hb`) не выходя из редактора.
- **lazygit:**
 - Открывай `lazygit` в отдельном сплите или табе для удобства.
 - Можно быстро делать `stage/unstage`, коммиты, пуши, даже не зная всех `git`-команд наизусть.
- **Перед удалением ветки** всегда проверяй, что она влита: `git branch --merged`.
- **Не забывай про .gitignore** — не коммить лишние файлы (`node_modules`, `.env` и т.д.).
- **Для сложных конфликтов** используй визуальные инструменты (`lazygit`, `VSCode`, `meld`).
- **Часто коммить — хорошо!** Мелкие коммиты проще откатывать и анализировать.
- **Пиши осмысленные сообщения к коммитам** — это поможет тебе и команде в будущем.

Видеоуроки и интерактивные тренажёры

YouTube

- [Learn Git In 15 Minutes \(ENG\)](#)
- [Git Tutorial for Beginners \(ENG\)](#)

- [Git and GitHub Tutorial for Beginners \[11 Hours\] \(ENG\)](#)

RuTube

- [PurpleSchool: Основы Git \(2024, RUS\)](#)

VK

- [Git для новичков — подборка видео](#)

Интерактивные тренажёры

- [GitHowTo — интерактивный курс \(RU\)](#)
- [Learn Git Branching — тренажёр](#)

Книги и гайды

- [Pro Git Book \(RU\)](#)
 - [Git Cheat Sheet \(ENG, PDF\)](#)
-

Ещё больше примеров команд Git

Работа с ветками

Показать все ветки

```
git branch -a
```

Переименовать текущую ветку

```
git branch -m new-name
```

Удалить локальную ветку

```
git branch -d old-branch
```

Удалить удалённую ветку

```
git push origin --delete old-branch
```

Сравнить две ветки

```
git diff main..feature
```

Работа с удалёнными репозиториями

Добавить новый remote

```
git remote add upstream https://github.com/other/repo.git
```

```
# Показать все remotes
git remote -v

# Получить изменения из другого remote
git fetch upstream

# Слить изменения из remote-ветки
git merge upstream/main
```

Слияния и rebase

```
# Слить ветку с fast-forward
git merge --ff-only feature

# Откатить merge-коммит
git revert -m 1 <merge-commit-hash>

# Интерактивный rebase последних 5 коммитов
git rebase -i HEAD~5

# Прервать rebase
git rebase --abort
```

Stash и восстановление

```
# Сохранить изменения с сообщением
git stash save "work in progress"

# Показать список stash
git stash list

# Посмотреть содержимое stash
git stash show -p stash@{0}

# Применить stash, но не удалять из списка
git stash apply stash@{0}

# Удалить все stash
git stash clear
```

Работа с файлами

```
# Удалить файл из индекса, но оставить на диске
git rm --cached file.txt

# Восстановить файл из последнего коммита
git checkout HEAD -- file.txt
```

```
# Показать изменения только в одном файле
git diff main.lua
```

История и поиск

```
# Показать историю только по файлу
git log --oneline -- file.txt
```

```
# Найти коммиты по автору
git log --author="Ivan"
```

```
# Найти коммиты по ключевому слову
git log --grep="bugfix"
```

```
# Показать изменения между двумя коммитами
git diff hash1..hash2
```

```
# Найти коммит, где появилась строка
git log -S"someFunction"
```

Теги и релизы

```
# Показать все теги
git tag
```

```
# Создать аннотированный тег
git tag -a v2.0 -m "Release 2.0"
```

```
# Отправить теги на сервер
git push --tags
```

```
# Удалить тег локально и на сервере
git tag -d v1.0
git push origin :refs/tags/v1.0
```

Cherry-pick и bisect

```
# Взять коммит из другой ветки
git cherry-pick <commit-hash>
```

```
# Начать bisect для поиска бага
git bisect start
git bisect bad
git bisect good <hash>
# ... тестируй, отмечай good/bad ...
git bisect reset
```

Алиасы и настройки

```
# Создать алиас для короткой команды
git config --global alias.co checkout

# Показать глобальные настройки
git config --global --list

# Изменить email для текущего репозитория
git config user.email "me@site.com"
```

.gitignore и очистка

```
# Добавить правило в .gitignore
echo 'node_modules/' >> .gitignore

git rm -r --cached node_modules/
git add .gitignore
git commit -m "ignore node_modules"

# Удалить все неотслеживаемые файлы и папки
git clean -fdx
```

Разное

```
# Показать короткую статистику по коммитам
git shortlog -sn

# Показать, кто внёс больше всего изменений
git log --pretty="%an" | sort | uniq -c | sort -nr | head

# Сравнить рабочую директорию с индексом
git diff

# Сравнить индекс с последним коммитом
git diff --cached

# Показать все файлы, изменённые в коммите
git show --name-only <commit-hash>
```

Подробная документация по git-плагинам для Neovim gumirus